randproj1

March 25, 2026

1 Project 1

Example using the sklearn library:

[1]:
```
# Main functions
%matplotlib widget

from scipy.io import wavfile
import array
import scipy
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, Markdown, Latex, Audio
from numpy import sqrt
from sklearn.decomposition import FastICA

plt.style.use('default')
plt.interactive(True)

REFERENCE_MIX = "./data0proj.wav"

SIGNAL0_FILE = "./sample-01.wav"
SIGNAL1_FILE = "./sample-03.wav"
SIGNAL2_FILE = "./sample-02.wav"
SIGNAL3_FILE = "./sample-04.wav"

default_mix = [[1, 1], [0.5, 2]]
default_noise_level = 0.05


def ReadWavFile(path):
    samplerate, signal_raw = wavfile.read(path)
    samples = np.array([float(s)/2**15 for s in signal_raw])
    n_samples = len(samples)
    times = np.linspace(0, (1/samplerate)*n_samples, n_samples)
    return samples, times, samplerate
def PlotAudio(samples, samplerate, title=""):
    times = np.linspace(0, (1/samplerate)*len(samples), len(samples))
```

```
    fig = plt.figure()
    plt.plot(times, samples)
    if title != "":
        plt.title(title)
    plt.show()
    display(Audio(data=samples, rate=samplerate))
def AddNoise(data, level, f = np.random.normal):
    level = np.max(data) * level
    s = data + (f(size=data.shape) * level)
    return s * abs(1/np.max(s)) # normalized

# quality - mean squared error
def DecompositionQuality(original, decomposed):
    diff = original-decomposed
    mean = np.mean(diff)
    score = np.mean(np.array([x**2 for x in diff]))
    return score, mean, diff # quality, mean, diff.

def PlotDecompositionQuality(original, decomposed, label):
    quality, mean, diff = DecompositionQuality(original, decomposed)
    var = np.var(diff)
    plt.figure()
    plt.plot(diff, label="difference" )
    plt.axhline(y=var, color="g", linestyle=":", label="Variance: " +␣
 ↪str(round(var, 4)))
    plt.title("Quality of " + label + " decomposition : " + str(quality))
    plt.legend()
    plt.show()
    return quality, mean, diff

# Assumption: the original signal will clearly have a corresponding decomposed␣
 ↪signal. This might be unwise.
def AssignDecomposed(originals, decomposed):
    assert len(originals) == len(decomposed)
    #return decomposed
    values = []
    i = 0
    for o in originals:
        ii = 0
        for d in decomposed:
            q, _, _ = DecompositionQuality(o, d)
            values.append((q, i, ii))
            ii += 1
        i += 1
    taken = []
    values = sorted(values, key=lambda v: v[0])
    slots = [None] * len(originals)
```

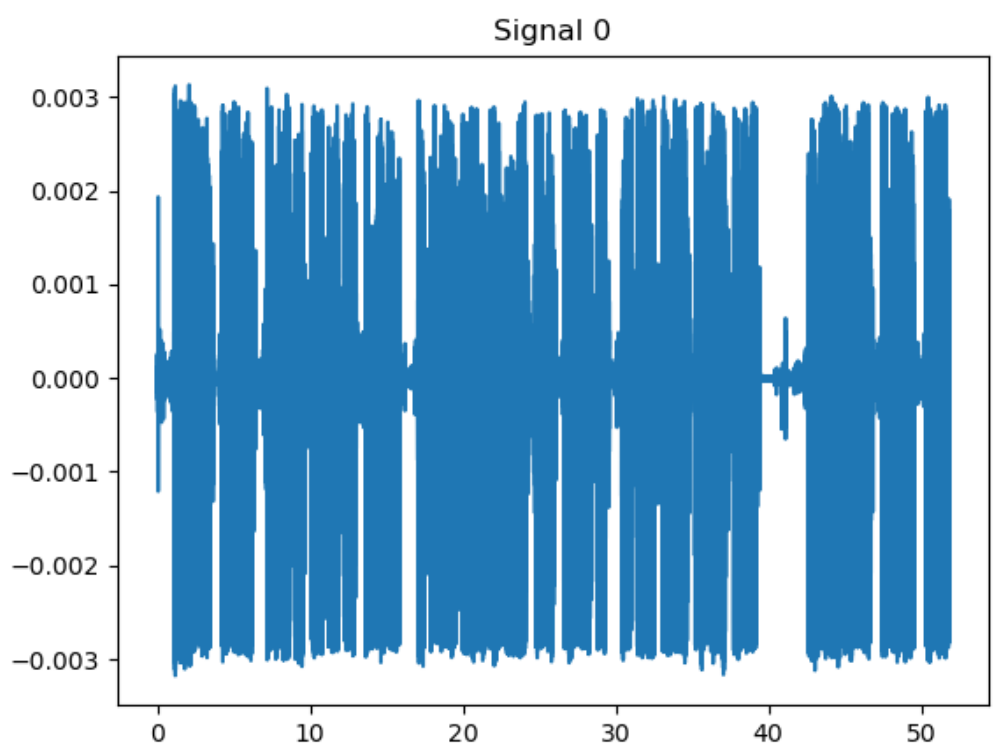
```
    nt = 0
    for it in values:
        if nt == len(originals):
            break
        if slots[it[1]] is not None:
            continue
        if it[2] in taken:
            continue

        slots[it[1]] = decomposed[it[2]]
        taken.append(it[2])
        nt += 1
    assert nt == len(originals)
    print(values)
    print(slots)
    return slots
```

[2]:
```
# Reaading and generating data

signal0, times, signal0_samplerate = ReadWavFile(SIGNAL0_FILE)
signal1, times, signal1_samplerate = ReadWavFile(SIGNAL1_FILE)
signal2, times, signal2_samplerate = ReadWavFile(SIGNAL2_FILE)
signal3, times, signal3_samplerate = ReadWavFile(SIGNAL3_FILE)

assert signal0_samplerate == signal1_samplerate
assert signal2_samplerate == signal3_samplerate
assert signal1_samplerate == signal2_samplerate

signal0 = signal0 / np.linalg.norm(signal0)
signal1 = signal1 / np.linalg.norm(signal1)
signal2 = signal2 / np.linalg.norm(signal2)
signal3 = signal3 / np.linalg.norm(signal3)

samplerate = signal0_samplerate

display(Markdown("## Signal 0:"))
PlotAudio(signal0, samplerate, "Signal 0")

display(Markdown("## Signal 1:"))
PlotAudio(signal1, samplerate, "Signal 1")

# add noise
S = np.c_[AddNoise(signal0, default_noise_level), AddNoise(signal1,␣
 ↪default_noise_level)]

S /= S.std(axis=0)  # Standardize data
```

```
A = np.array(default_mix)  # Mixing matrix
X = np.dot(S, A.T)  # Generate observations

mic0 = X[:,0]
mic1 = X[:,1]

PlotAudio(mic0, samplerate, "Microphone 0 reading")
PlotAudio(mic1, samplerate, "Microphone 1 reading")
```

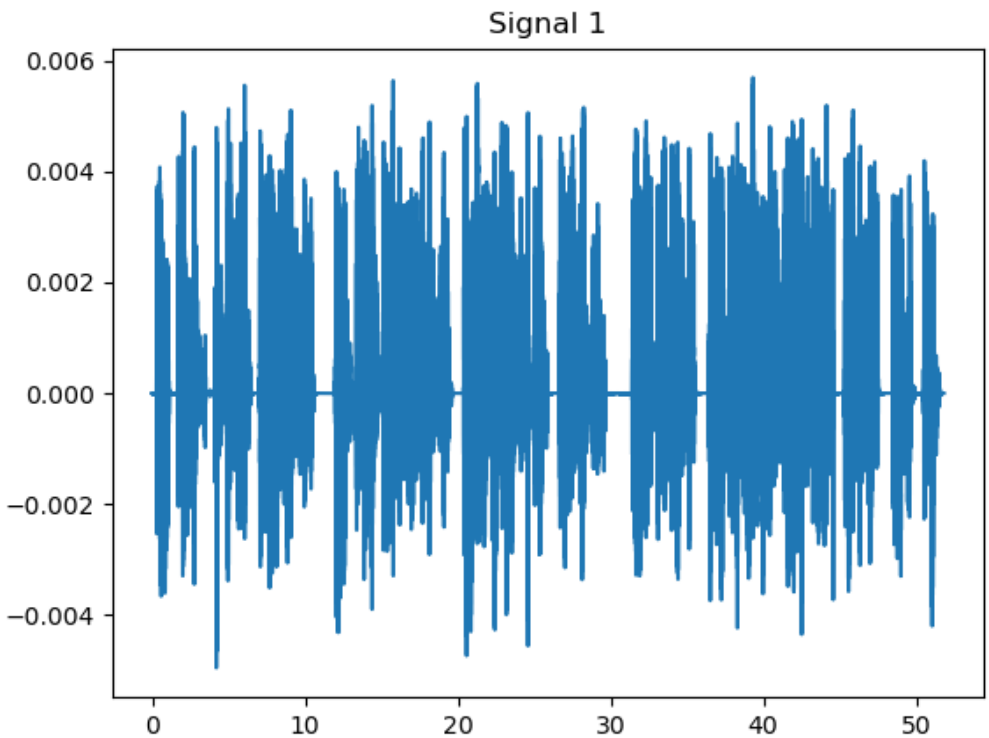
```
/tmp/ipykernel_195983/2566091943.py:28: WavFileWarning: Chunk (non-data) not
understood, skipping it.
  samplerate, signal_raw = wavfile.read(path)
```

1.1 Signal 0:

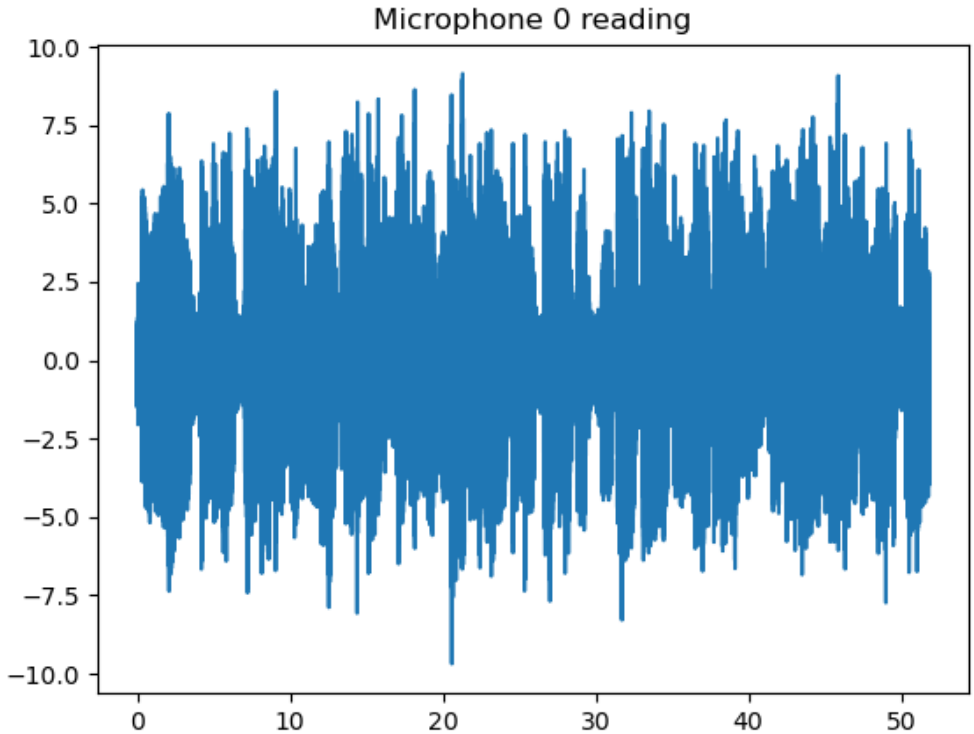



```
<IPython.lib.display.Audio object>
```

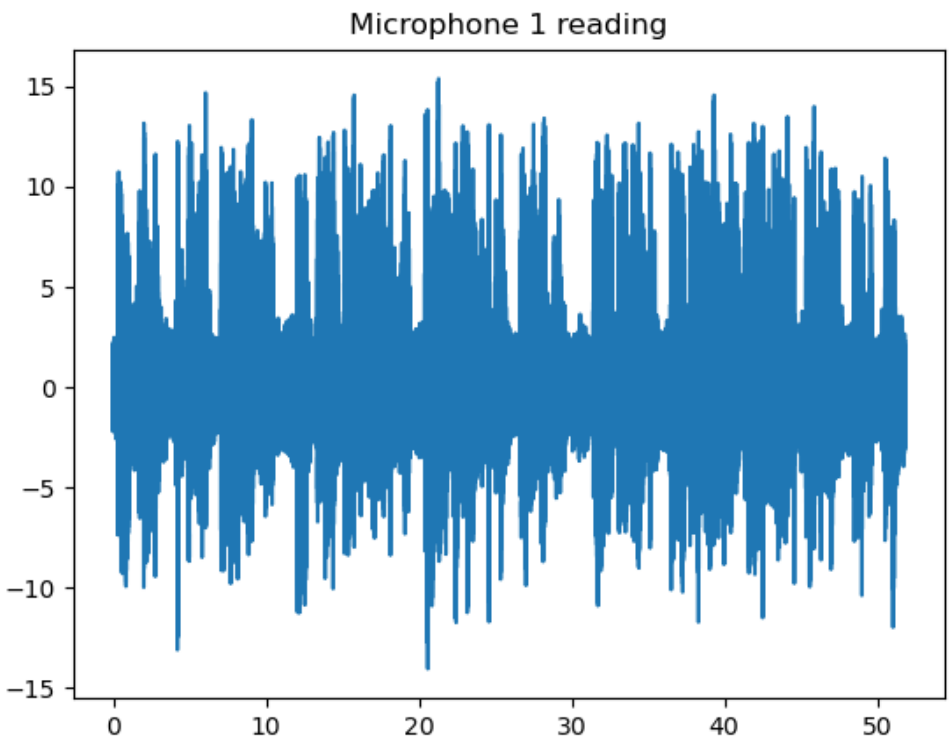
1.2 Signal 1:




```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```

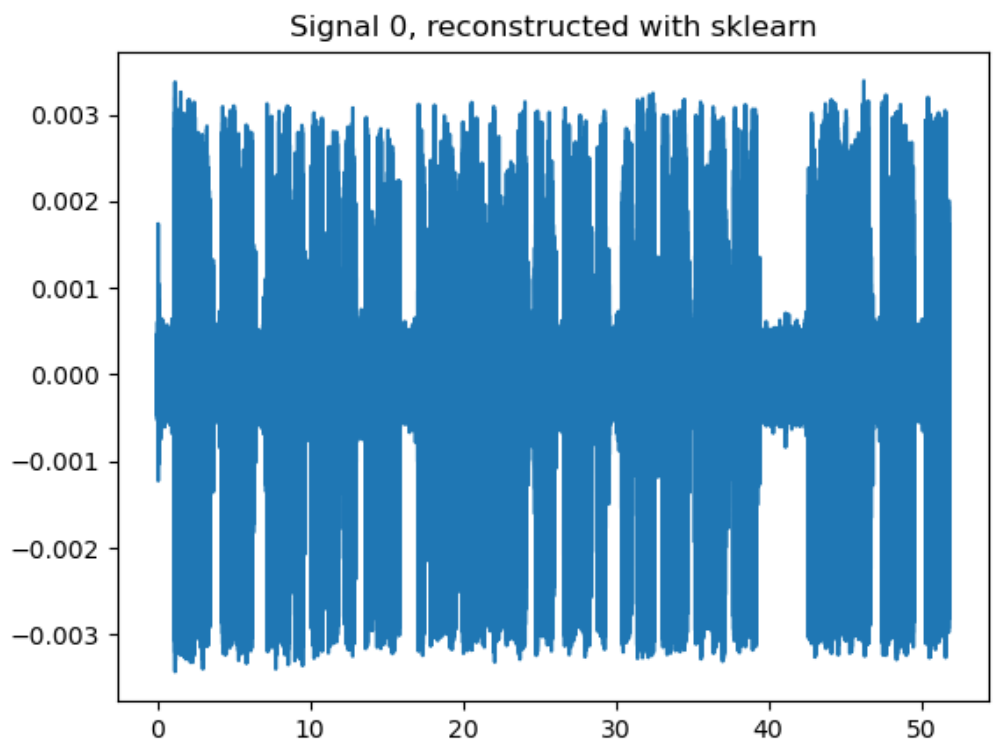



```
<IPython.lib.display.Audio object>
```

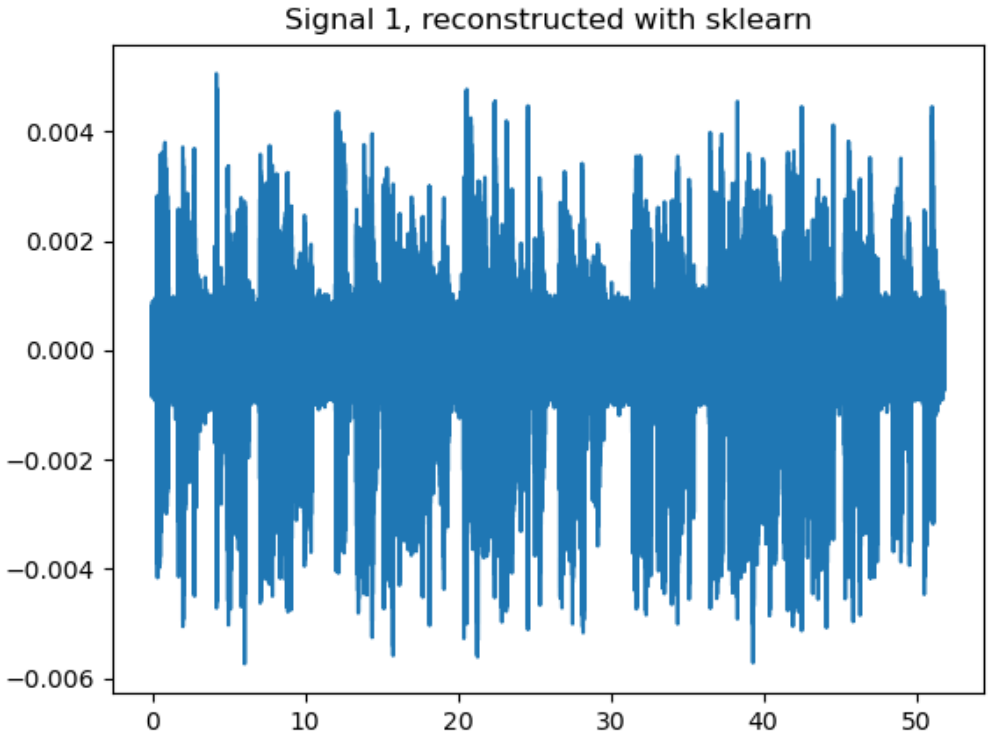
[3]:
```
# Reference test

ica = FastICA(n_components=2, whiten="arbitrary-variance")
S_ = ica.fit_transform(X)  # Reconstruct signals
A_ = ica.mixing_  # Get estimated mixing matrix

assigned = AssignDecomposed([signal0, signal1], [S_[:, 0] / np.linalg.norm(S_[:
↪, 0]), S_[:, 1] / np.linalg.norm(S_[:, 1])])
reconstructed0 = assigned[0]
reconstructed1 = assigned[1]
assigned = None

PlotAudio(reconstructed0, samplerate, "Signal 0, reconstructed with sklearn")

PlotAudio(reconstructed1, samplerate, "Signal 1, reconstructed with sklearn")

display(Markdown(f"Reference quality for mixing matrix {default_mix},␣
↪noise={default_noise_level}"))
```

```
qref0, _, _ = PlotDecompositionQuality(signal0, reconstructed0, "signal 0")
qref1, _, _ = PlotDecompositionQuality(signal1, reconstructed1, "signal 1")

# conserve RAM
reconstructed0 = None
reconstructed1 = None
```

```
[(np.float64(2.3754379992932136e-08), 0, 0),
(np.float64(1.2076523083529884e-06), 1, 0), (np.float64(1.208720931316825e-06),
0, 1), (np.float64(2.3383552745509213e-06), 1, 1)]
[array([-0.00017689,  0.00022489, -0.00011451, …,  0.00084268,
        0.00078717,  0.00056754], shape=(1658398,)), array([-1.38133598e-04,
1.99106576e-04, -9.39248298e-06, …,
        1.21411914e-04, -1.19858096e-04, -1.03260054e-04],
      shape=(1658398,))]
```

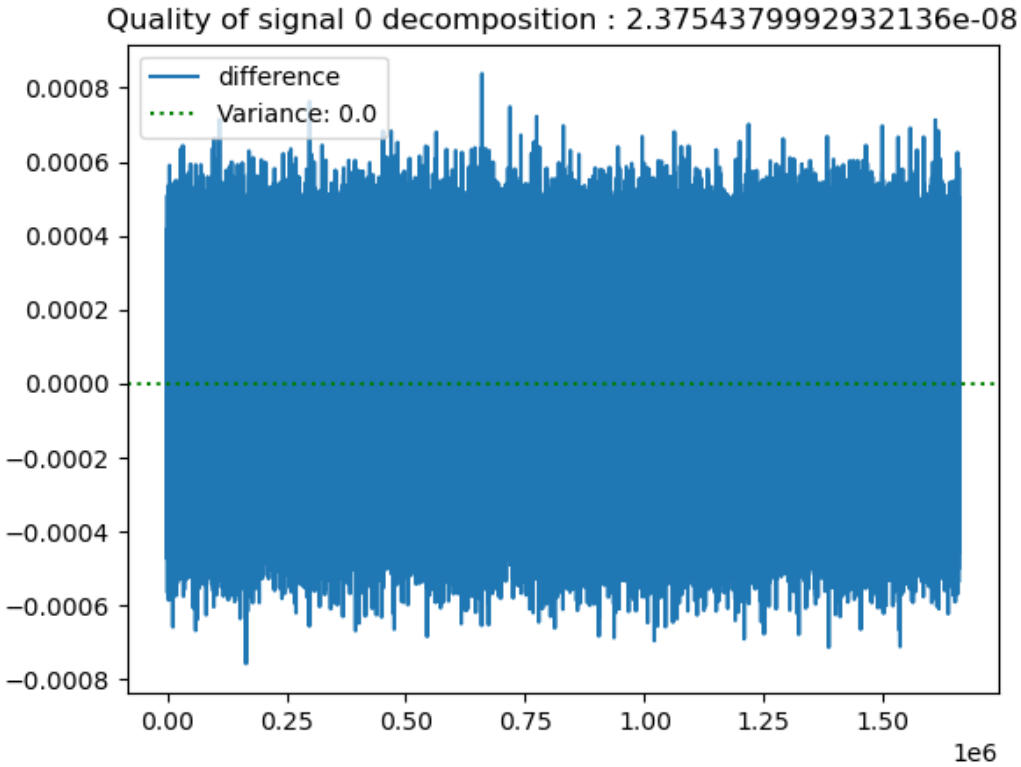


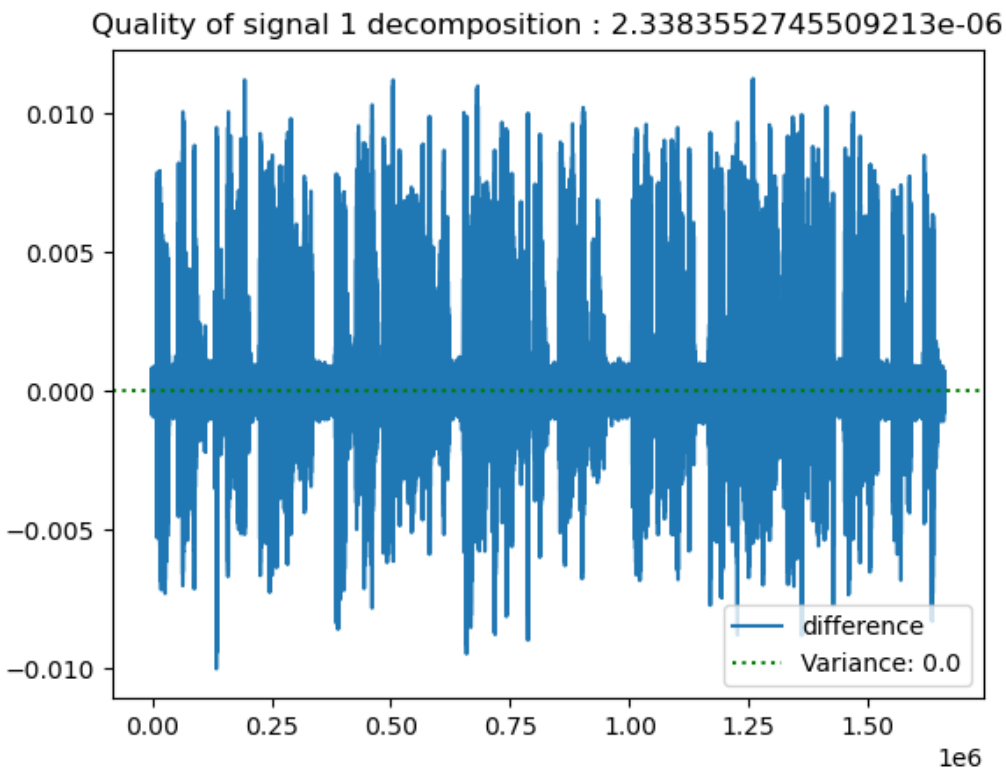

```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```

Reference quality for mixing matrix [[1, 1], [0.5, 2]], noise=0.05


Quality of signal 0 decomposition : 2.3754379992932136e-08
difference
Variance: 0.0
0.0008
0.0006
0.0004
0.0002
0.0000
−0.0002
−0.0004
−0.0006
−0.0008
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1e6



2 Implementation, basic test

2.1 Whitening:

X - input

X_w - Whitened output

C - covariance matrix of X

E - eigen vectors of C

D - eigen values of C

$X_w = D^{-1/2} E^T X$

[4]:
```
from numpy import nan
import sys
import time
epsilon = sys.float_info.epsilon
def g(data):
    return np.tanh(data)
def g_prime(data):
```

```
    return 1 - np.tanh(data)**2
def g_both(data):
    g_ret = np.tanh(data)
    g_prime_ret = 1 - g_ret**2
    return g_ret, g_prime_ret

def FastICA(readings: np.ndarray, maxiter=1000, tolerance = 1e-7, showtime =␣
 ↪True):
    start = time.time()
    n = readings.shape[0]
    nsamples = readings.shape[1]
    signals = []

    # center
    for i in range(0, n):
        signal : np.ndarray = readings[i, :]
        signal -= np.mean(signal) # zero the mean
        signals.append(signal)
    S = np.array(signals)

    # whiten the input data
    cov = np.cov(S)
    evals, evecs = np.linalg.eigh(cov)
    inv = np.diag(1.0 / np.sqrt(evals + epsilon))
    #whitened input
    Sw = inv @ evecs.T @ S
    iters = 0

    onescol = np.array([[1]] * nsamples)
    Wmats = []
    for p in range(0, n):
        Wp = np.random.uniform(size=(n, 1))
        i = 0
        while i < maxiter:
            WX = Wp.T @ Sw
            gW, gW_prime = g_both(WX)
            L = (1/nsamples)*(Sw @ gW.T)
            R = (1/nsamples) * (gW_prime @ onescol) @ Wp.T
            Wnew = L - R.T
            for w in Wmats:
                w = np.array([w]).T
                Wnew -= (Wnew.T @ w) * w
            Wnew = Wnew / np.linalg.norm(Wnew)
            dif = 1 - abs((Wp.T @ Wnew).item())
            if np.isnan(dif):
                Wp = np.random.uniform(size=(n, 1))
                print("Invalid iteration, got nan dif!")
```

```
            else:
                Wp = Wnew
            print(f"it: {i} dif: {dif}")
            if dif < tolerance:
                break
            i += 1
        if i == maxiter:
            print("Maximum number of iterations exceeded!")
        Wmats.append(np.array([Wp[i][0] for i in range(0, n)]))
        iters += i
    Wmats = np.array(Wmats)
    res = Wmats @ Sw
    # normalize the samples
    for i in range(0, res.shape[0]):
        res[i] = res[i, :] / np.linalg.norm(res[i, :])

    end = time.time()
    elapsed = end-start
    if showtime:
        display(Markdown(f"### Decomposed {n} inputs into {n} outputs in␣
↪{iters} iterations, elapsing {elapsed}s"))

    return res, iters
```

3 Test I: two signals

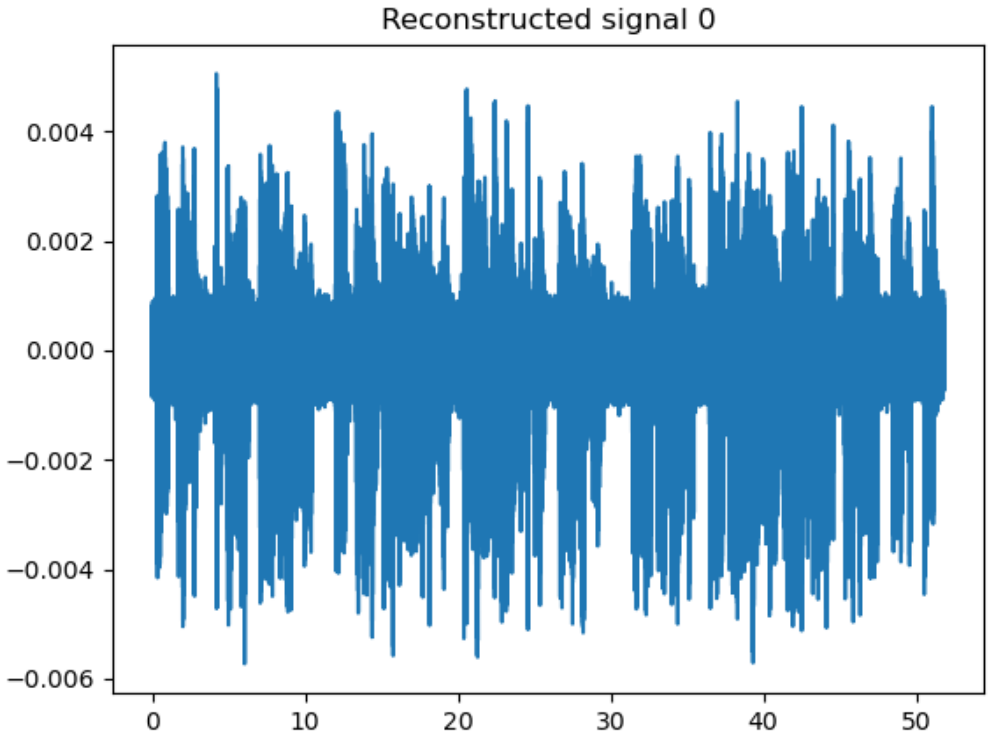
[5]:
```
res, n_iters = FastICA(np.array([mic0, mic1]))

assigned = AssignDecomposed([signal0, signal1], [res[0, :], res[1, :]])
r0 = assigned[0]
r1 = assigned[1]
assigned = None

PlotAudio(r0, samplerate, "Reconstructed signal 0")

PlotAudio(r1, samplerate, "Reconstructed signal 1")

display(Markdown(f"Quality for mixing matrix {default_mix},␣
↪noise={default_noise_level}"))


q0, _, _ = PlotDecompositionQuality(signal0, r0, "signal 0")
display(Markdown(f"Reference quality for signal 0 = {qref0}"))
qdif0 = q0 - qref0
display(Markdown(f"Achieved quality for signal 0 = {q0} ({"+" if qdif0 > 0 else␣
↪""}{qdif0})"))
```

```
q1, _, _ = PlotDecompositionQuality(signal1, r1, "signal 1")
display(Markdown(f"Reference quality for signal 1 = {qref1}"))
qdif1 = q1 - qref1
display(Markdown(f"Achieved quality for signal 1 = {q1} ({"+" if qdif1 > 0 else␣
↪""}{qdif1})"))

r0 = None
r1 = None
```

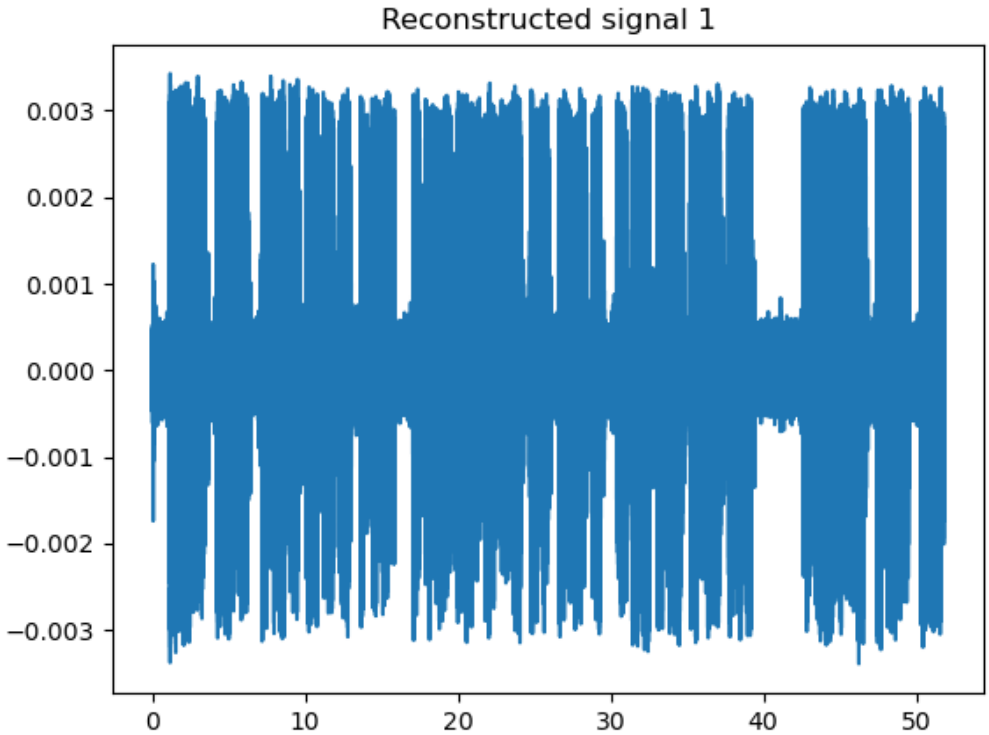
```
it: 0 dif: 0.5313708788605337
it: 1 dif: 5.001490327205538e-05
it: 2 dif: 5.7135740361147214e-08
it: 0 dif: 0.6447227715212085
it: 1 dif: 1.1102230246251565e-16
```

3.0.1 Decomposed 2 inputs into 2 outputs in 3 iterations, elapsing 0.7060749530792236s

```
[(np.float64(1.2019211541377938e-06), 1, 1),
(np.float64(1.2112190641431116e-06), 0, 0), (np.float64(2.338349219362546e-06),
1, 0), (np.float64(2.388203442374524e-06), 0, 1)]
[array([-1.37759500e-04,  1.98630916e-04, -9.15049246e-06, …,
        1.19630994e-04, -1.21521184e-04, -1.04459069e-04],
      shape=(1658398,)), array([ 0.00017718, -0.00022531,  0.00011453, …,
-0.00084294,
       -0.00078692, -0.00056732], shape=(1658398,))]
```

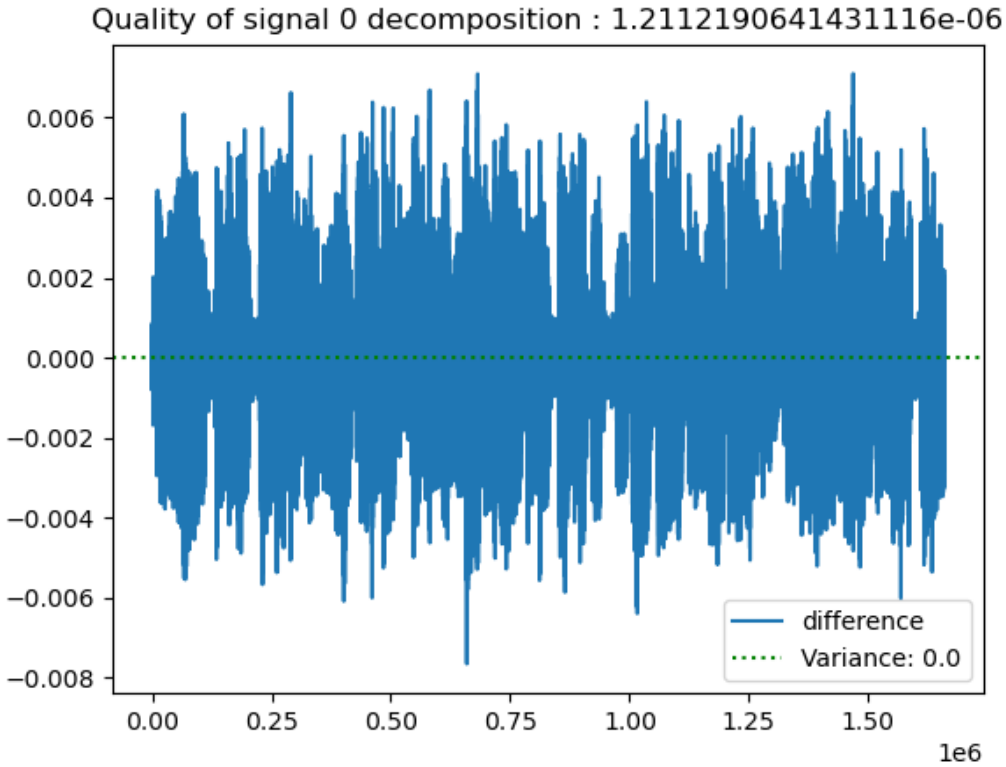



```
<IPython.lib.display.Audio object>
```



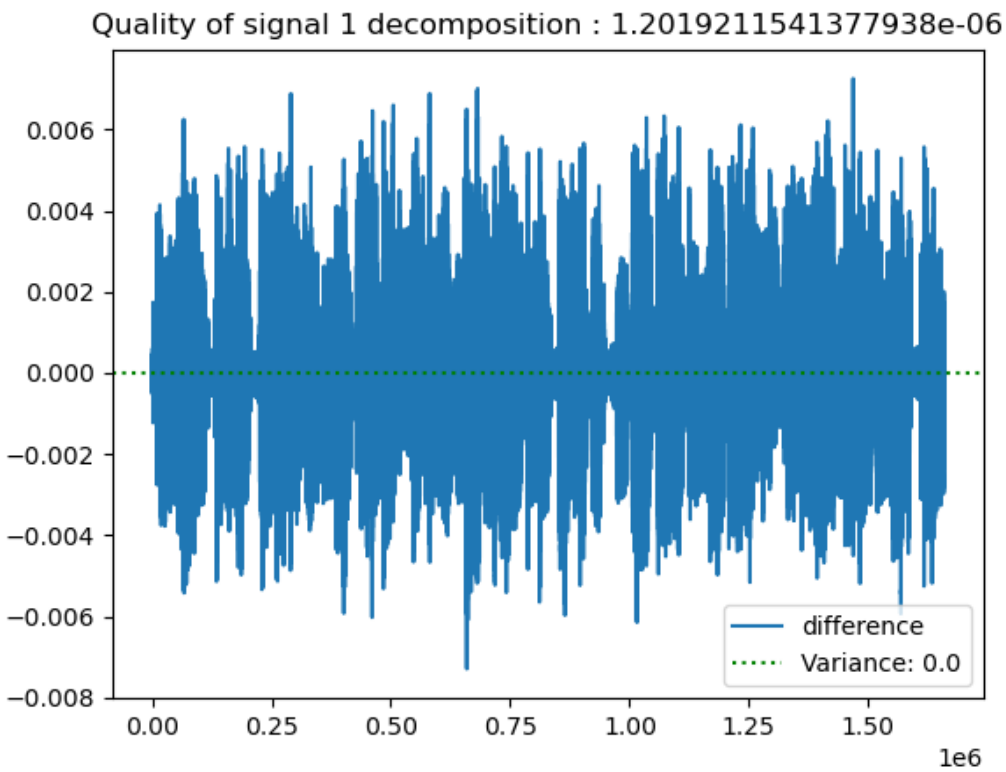

```
<IPython.lib.display.Audio object>
```

Quality for mixing matrix [[1, 1], [0.5, 2]], noise=0.05



Reference quality for signal 0 = 2.3754379992932136e-08

Achieved quality for signal 0 = 1.2112190641431116e-06 (+1.1874646841501794e-06)



Reference quality for signal 1 = 2.3383552745509213e-06

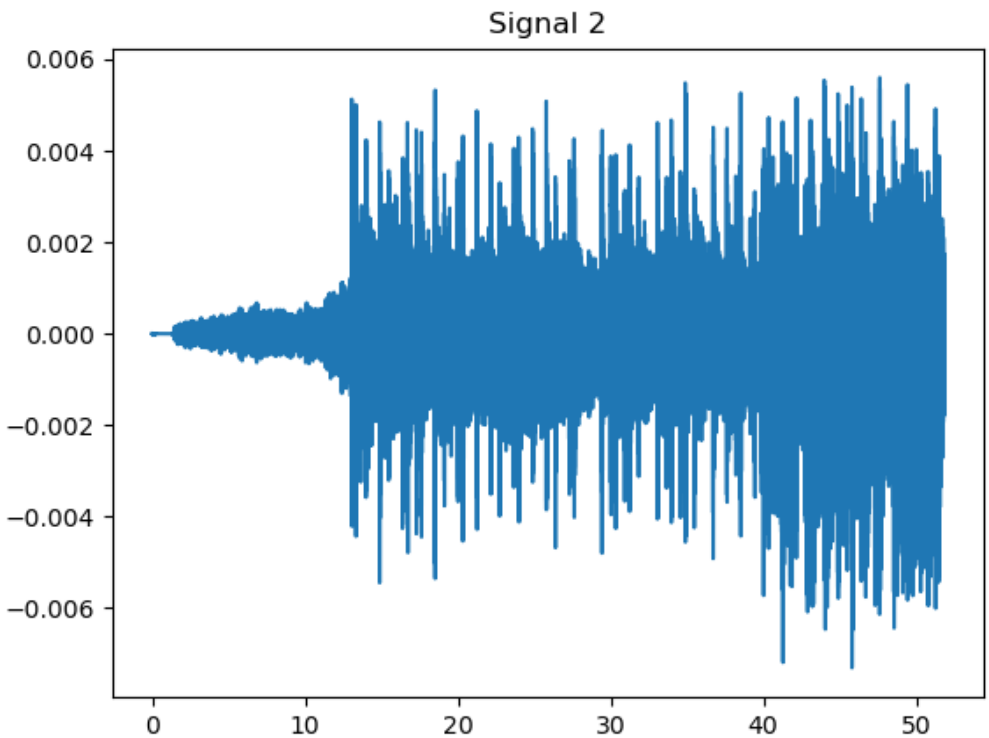
Achieved quality for signal 1 = 1.2019211541377938e-06 (-1.1364341204131275e-06)

4 Test II - 3 signals

[6]:
```
display(Markdown("## Signal 2:"))
PlotAudio(signal2, samplerate, "Signal 2")


default_mix = [[1, 1, 0.5], [0.5, 2, 1], [0.25, 1.5, 1.1]]
default_noise_level = 0.025

# add noise
S = np.c_[AddNoise(signal0, default_noise_level), AddNoise(signal1,␣
↪default_noise_level), AddNoise(signal2, default_noise_level)]

S /= S.std(axis=0)  # Standardize data

A = np.array(default_mix)  # Mixing matrix
X = np.dot(S, A.T)  # Generate observations
```

```
mic0 = X[:,0]
mic1 = X[:,1]
mic2 = X[:,2]


display(Markdown("Mic 0:"))
display(Audio(data=mic0, rate=samplerate))

display(Markdown("Mic 1:"))
display(Audio(data=mic1, rate=samplerate))

display(Markdown("Mic 2:"))
display(Audio(data=mic2, rate=samplerate))

res, n_iters = FastICA(np.array([mic0, mic1, mic2]), tolerance=1e-8)

assigned = AssignDecomposed([signal0, signal1, signal2], [res[0, :], res[1, :],␣
 ↪res[2, :]])
r0 = assigned[0]
r1 = assigned[1]
r2 = assigned[2]
assigned = None

PlotAudio(r0, samplerate, "Reconstructed signal 0")
PlotAudio(r1, samplerate, "Reconstructed signal 1")
PlotAudio(r2, samplerate, "Reconstructed signal 2")
```

4.1 Signal 2:




```
<IPython.lib.display.Audio object>
```

Mic 0:

```
<IPython.lib.display.Audio object>
```

Mic 1:

```
<IPython.lib.display.Audio object>
```

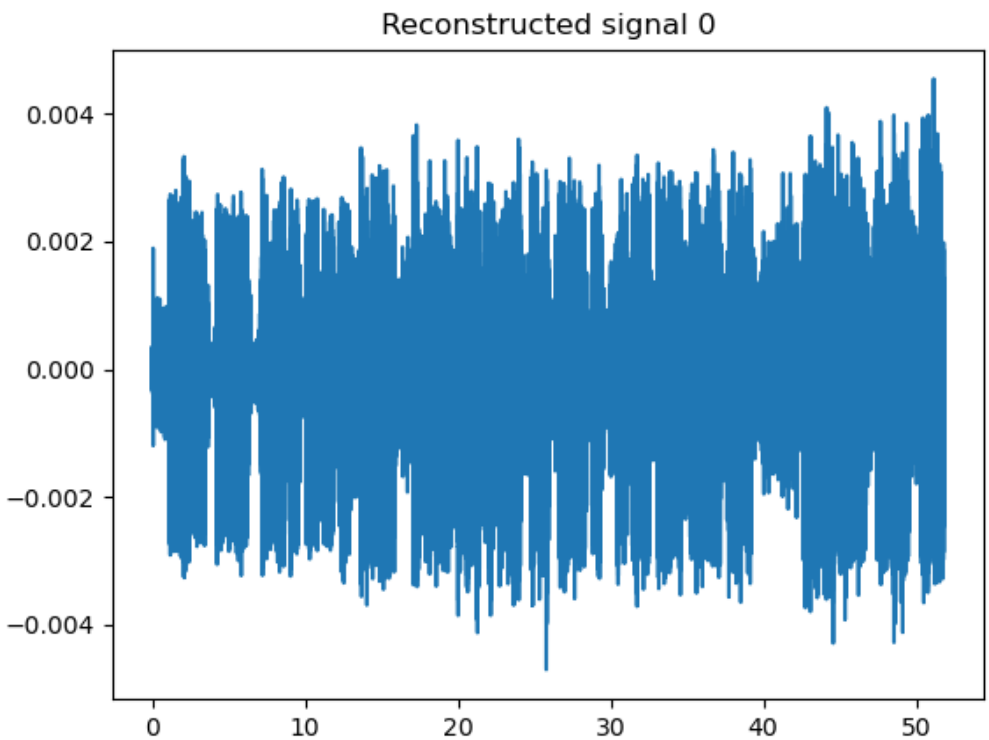
Mic 2:

```
<IPython.lib.display.Audio object>
```

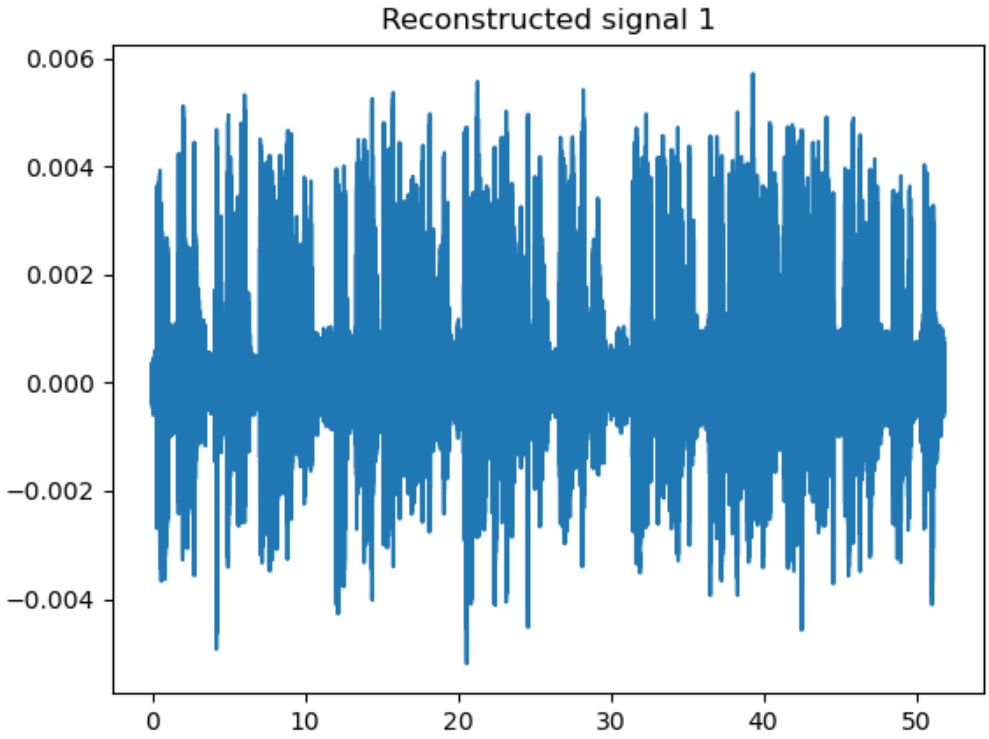
```
it: 0 dif: -0.21716283215423537
it: 0 dif: 0.036801453477589785
it: 1 dif: 0.0025695709420837654
it: 2 dif: 1.9352919160997573e-05
it: 3 dif: 2.3685633376846482e-07
it: 4 dif: 3.0430772346434765e-09
it: 0 dif: 0.7730569731298499
it: 1 dif: 2.220446049250313e-16
```

4.1.1 Decomposed 3 inputs into 3 outputs in 5 iterations, elapsing 1.2640328407287598s

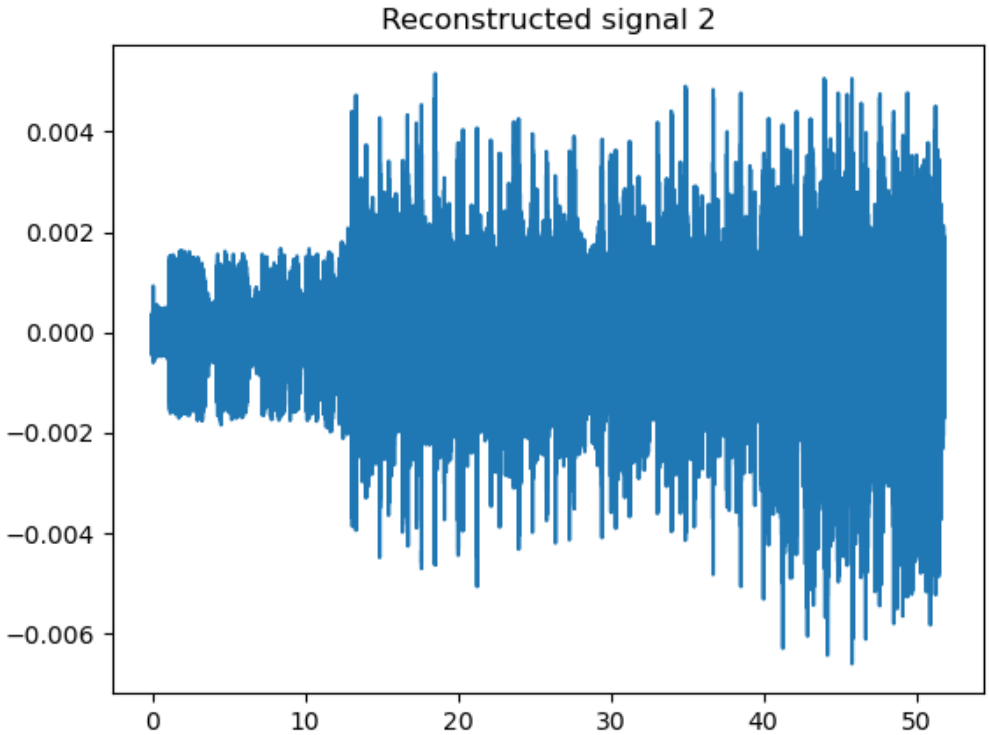
```
[(np.float64(5.5709875678961544e-08), 1, 1), (np.float64(1.383935045019471e-07),
2, 2), (np.float64(1.5996063574507375e-07), 0, 0),
(np.float64(6.819750277646093e-07), 0, 2), (np.float64(9.160857674271969e-07),
1, 0), (np.float64(1.0907258456628772e-06), 2, 1),
(np.float64(1.1973962751363428e-06), 1, 2), (np.float64(1.472466178712828e-06),
0, 1), (np.float64(1.7116058868681597e-06), 2, 0)]
[array([ 6.28327235e-05,  2.28463697e-05, -1.16439406e-04, …,
        5.89661077e-04,  2.63874253e-04,  3.99251467e-04],
      shape=(1658398,)), array([-1.96641388e-04, -3.57589231e-05,
-4.43099986e-05, …,
       -2.48570515e-04,  3.35168586e-05, -9.18944865e-05],
      shape=(1658398,)), array([-1.29160355e-04,  2.34616797e-04,
7.54081940e-05, …,
        7.30964510e-04,  8.57152971e-04,  5.38797480e-04],
      shape=(1658398,))]
```




```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```



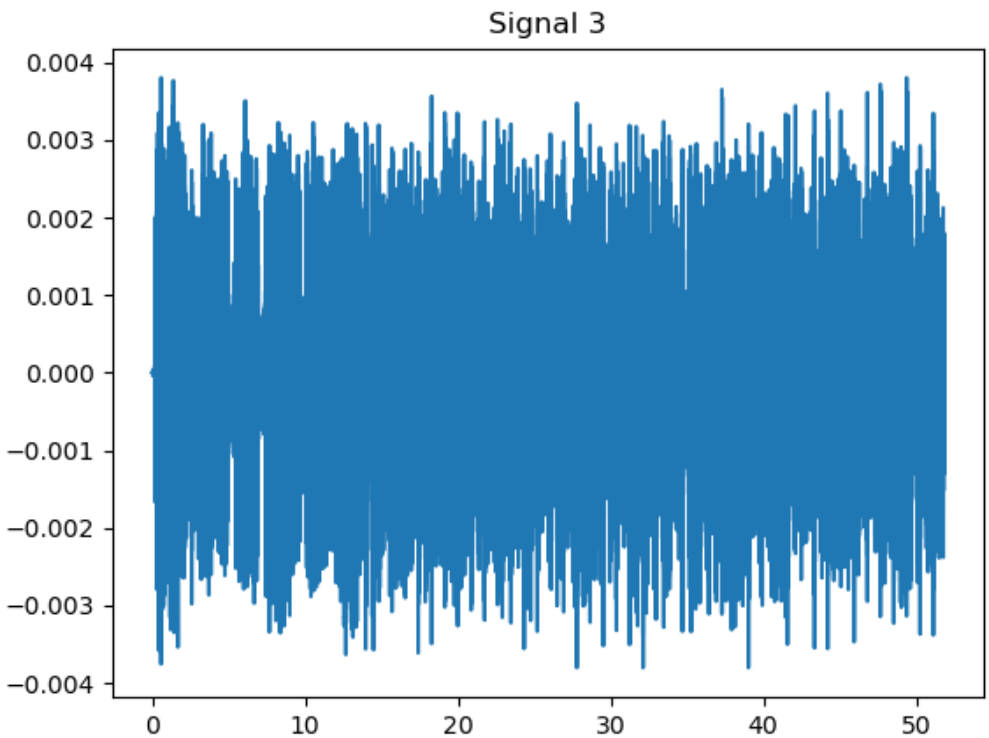

```
<IPython.lib.display.Audio object>
```

5 Test III - 4 signals

[7]:
```
display(Markdown("## Signal 3:"))
PlotAudio(signal3, samplerate, "Signal 3")


default_mix = [[0.1, 1, 1, 0.5], [3, 0.5, 2, 1], [1.1, 0.25, 1.5, 1.1], [0.9, 1.
↪8, 0.13, 0.3]]
default_noise_level = 0.0125

# add noise
S = np.c_[AddNoise(signal0, default_noise_level), AddNoise(signal1,␣
↪default_noise_level), AddNoise(signal2, default_noise_level),␣
↪AddNoise(signal3, default_noise_level)]

S /= S.std(axis=0)  # Standardize data

A = np.array(default_mix)  # Mixing matrix
```

```
X = np.dot(S, A.T)  # Generate observations

mic0 = X[:,0]
mic1 = X[:,1]
mic2 = X[:,2]
mic3 = X[:,3]

S = None


display(Markdown("Mic 0:"))
display(Audio(data=mic0, rate=samplerate))

display(Markdown("Mic 1:"))
display(Audio(data=mic1, rate=samplerate))

display(Markdown("Mic 2:"))
display(Audio(data=mic2, rate=samplerate))

res, n_iters = FastICA(np.array([mic0, mic1, mic2, mic3]), tolerance=1e-9)

assigned = AssignDecomposed([signal0, signal1, signal2, signal3], [res[0, :],␣
 ↪res[1, :], res[2, :], res[3, :]])
r0 = assigned[0]
r1 = assigned[1]
r2 = assigned[2]
r3 = assigned[3]
assigned = None
res = None

PlotAudio(r0, samplerate, "Reconstructed signal 0")
PlotAudio(r1, samplerate, "Reconstructed signal 1")
PlotAudio(r2, samplerate, "Reconstructed signal 2")
PlotAudio(r3, samplerate, "Reconstructed signal 2")
```

5.1 Signal 3:




```
<IPython.lib.display.Audio object>
```

Mic 0:

```
<IPython.lib.display.Audio object>
```

Mic 1:

```
<IPython.lib.display.Audio object>
```

Mic 2:

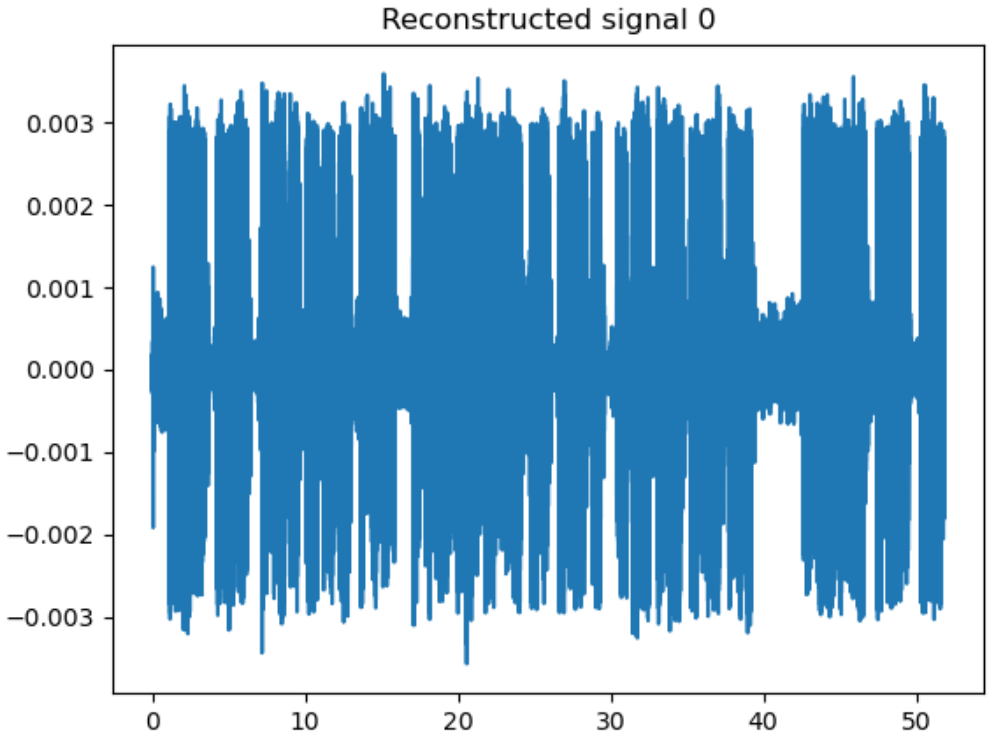
```
<IPython.lib.display.Audio object>
```

```
it: 0 dif: -0.01762260677227001
it: 0 dif: 0.9645019633723999
it: 1 dif: 0.12754916548448958
it: 2 dif: 0.043666712990494516
it: 3 dif: 0.0006238817867459945
it: 4 dif: 3.428767431312707e-06
it: 5 dif: 1.8365864828950862e-08
it: 6 dif: 9.828882152618235e-11
it: 0 dif: 0.9434953295381024
```

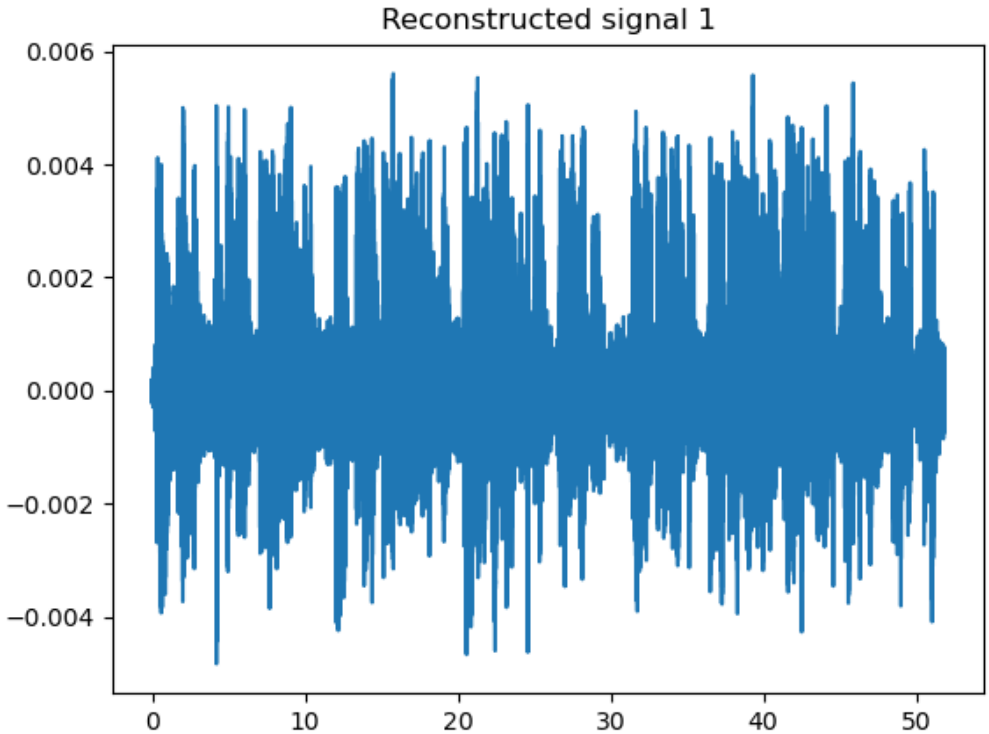
```
it: 1 dif: 0.003610655847046451
it: 2 dif: 7.671137169928954e-08
it: 3 dif: 1.8807932988806897e-10
it: 0 dif: 0.8230841187496976
it: 1 dif: 0.0
```

5.1.1 Decomposed 4 inputs into 4 outputs in 10 iterations, elapsing 1.4163331985473633s

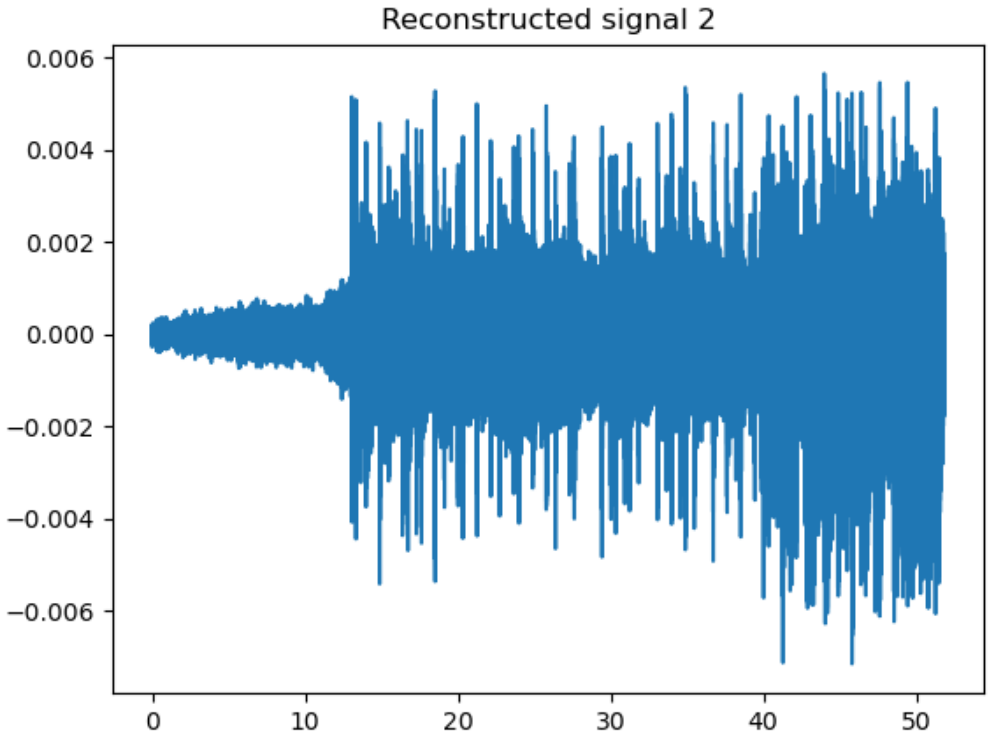
```
[(np.float64(7.0923130036568766e-09), 2, 3), (np.float64(7.992944455966186e-08),
3, 2), (np.float64(1.0340294375489212e-07), 1, 0),
(np.float64(7.888052200588832e-07), 3, 0), (np.float64(9.934073230577878e-07),
0, 0), (np.float64(1.0085285584011218e-06), 1, 1),
(np.float64(1.1276611019401435e-06), 3, 1), (np.float64(1.1345004816610902e-06),
2, 0), (np.float64(1.197181431846031e-06), 2, 1),
(np.float64(1.2073345012166238e-06), 0, 2), (np.float64(1.2101205627699114e-06),
0, 3), (np.float64(1.218455380016669e-06), 2, 2),
(np.float64(1.2341768969479745e-06), 3, 3), (np.float64(1.2816723891790878e-06),
1, 3), (np.float64(1.632482694551358e-06), 1, 2),
(np.float64(2.391524130715916e-06), 0, 1)]
[array([-4.58619680e-05,  2.89036788e-05, -6.19685607e-06, …,
       -8.24153047e-04, -7.49437540e-04, -5.90629528e-04],
      shape=(1658398,)), array([-3.45944637e-05,  1.07828524e-05,
8.39381439e-05, …,
        5.63930140e-04,  4.84200792e-04,  2.99273932e-04],
      shape=(1658398,)), array([-6.02377954e-06, -1.38016860e-05,
-9.28772395e-07, …,
        3.50281759e-04,  1.80463948e-04,  4.64748498e-04],
      shape=(1658398,)), array([ 1.10132794e-04, -7.42715656e-05,
-8.68354773e-06, …,
        8.94759603e-04,  8.63281587e-04,  6.59088568e-04],
      shape=(1658398,))]
```




```
<IPython.lib.display.Audio object>
```

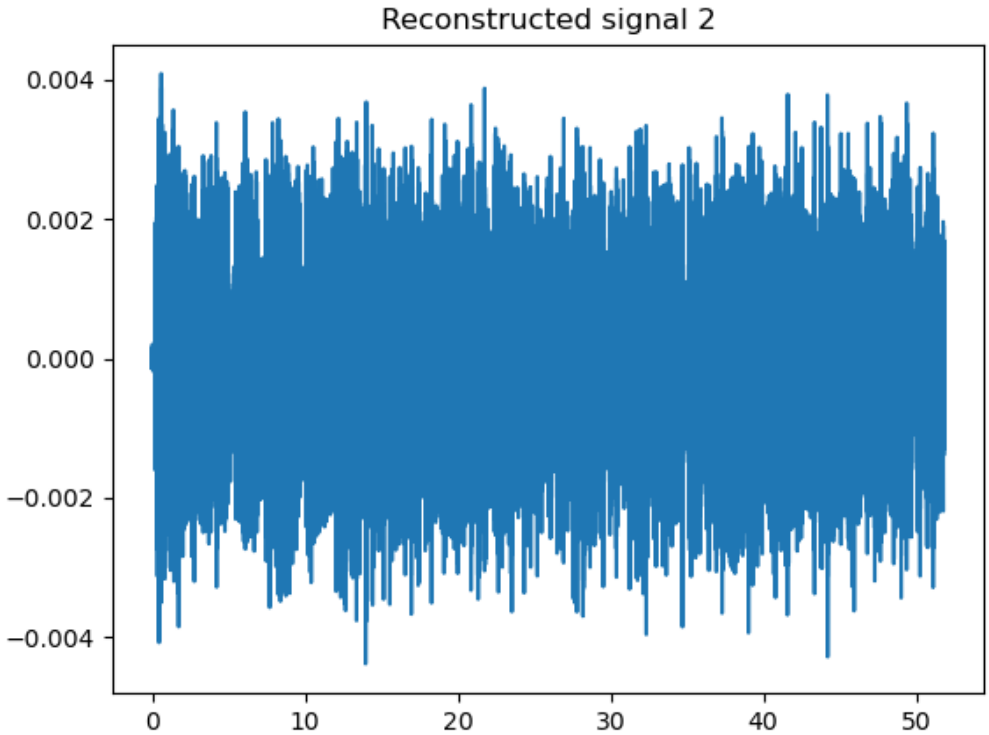



```
<IPython.lib.display.Audio object>
```




```
<IPython.lib.display.Audio object>
```

```
/tmp/ipykernel_195983/2566091943.py:35: RuntimeWarning: More than 20 figures
have been opened. Figures created through the pyplot interface
(`matplotlib.pyplot.figure`) are retained until explicitly closed and may
consume too much memory. (To control this warning, see the rcParam
`figure.max_open_warning`). Consider using `matplotlib.pyplot.close()`.
  fig = plt.figure()
```




```
<IPython.lib.display.Audio object>
```

[7]: